

What is kagent?#

kagent is an innovative AI agent platform designed specifically for Kubernetes environments. It empowers developers and operations teams to create intelligent, autonomous agents that can monitor, manage, and automate complex Kubernetes workloads using the power of large language models (LLMs).

kagent was created at [Solo.io](https://solo.io) in 2025 and is a [Cloud Native Computing Foundation](https://cloudnativecomputing.org) sandbox project.

Key Features#

- **AI-Powered Automation** - Create intelligent agents that understand natural language and can perform complex Kubernetes operations
- **Multi-Provider Support** - Works with OpenAI, Anthropic, Google Vertex AI, Azure OpenAI, Ollama, and custom models
- **Tool Integration** - Supports Model Context Protocol (MCP) tools, built-in Kubernetes tools, and custom HTTP tools
- **Agent-to-Agent Communication** - Enable sophisticated workflows through A2A (Agent-to-Agent) interactions
- **Comprehensive Observability** - Built-in tracing and monitoring to understand agent behavior and performance
- **Cloud Native** - Designed from the ground up to run natively in Kubernetes environments

Why Choose kagent?#

Whether you're looking to automate routine operations, implement intelligent monitoring, or create sophisticated multi-agent workflows, kagent provides the tools and framework to bring AI to your Kubernetes infrastructure. Start with simple automation and scale to complex, intelligent systems that can reason about your cluster's state and make informed decisions.

Creating a BYO Agent resource#

Now that you have your own custom agent image, you can create a BYO Agent resource for kagent to manage. You will need a Serper API Key that you can get for free [from their website](https://serper.dev). Serper is a Google Search API used by most CrewAI examples and tutorials, but you can also plug in your own tools.

1. Save the API keys for your LLM provider and web search service in environment variables.

```
export OPENAI_API_KEY=your-openai-api-key-here
```

```
export SERPER_API_KEY=your-serper-api-key-here
```

2. Create secrets with the API keys.

```
kubectl create secret generic kagent-openai -n kagent \
--from-literal=OPENAI_API_KEY=$OPENAI_API_KEY \
```

```
--dry-run=client -o yaml | kubectl apply -f -
```

```
kubectl create secret generic kagent-serper -n kagent \
```

```
--from-literal=SERPER_API_KEY=$SERPER_API_KEY \
```

```
--dry-run=client -o yaml | kubectl apply -f -
```

3. Create a BYO Agent resource.

```
kubectl apply -f - <<EOF
```

```
apiVersion: kagent.dev/v1alpha2
```

```
kind: Agent
```

```
metadata:
```

```
  name: research-crew
```

```
  namespace: kagent
```

```
spec:
```

```
  description: A research crew with multiple specialized agents for web research and analysis.
```

```
  type: BYO
```

```
  byo:
```

```
    deployment:
```

```
      image: ghcr.io/research-crew:latest
```

```
      env:
```

```
        - name: OPENAI_API_KEY
```

```
          valueFrom:
```

```
            secretKeyRef:
```

```
              name: kagent-openai
```

```
              key: OPENAI_API_KEY
```

```
        - name: SERPER_API_KEY
```

```
          valueFrom:
```

```
            secretKeyRef:
```

```
              name: kagent-serper
```

```
              key: SERPER_API_KEY
```

```
EOF
```

Testing the A2A endpoint#

The A2A endpoint is exposed on the port 8083 of the kagent controller service.

1. Enable port-forwarding on the kagent-controller service.

Note that you could also expose the A2A endpoint publicly by using a gateway.

```
kubectrl port-forward svc/kagent-controller 8083:8083 -n kagent
```

2. To test that the agent is available and has an agent card, send a request to the `.well-known/agent.json` endpoint. Note the API endpoint follows the pattern `/api/a2a/{namespace}/{agent-name}/.well-known/agent.json`.

```
curl localhost:8083/api/a2a/kagent/research-crew/.well-known/agent.json
```

Example output: This JSON object describes the agent as per the [A2A protocol](#).

```
{
  "name": "research_crew",
  "description": "A research crew with multiple specialized agents for web research and analysis.",
  "url": "http://127.0.0.1:8083/api/a2a/kagent/research-crew/",
  "version": "",
  "capabilities": {
    "streaming": true,
    "pushNotifications": false,
    "stateTransitionHistory": true
  },
  "defaultInputModes": ["text"],
  "defaultOutputModes": ["text"],
  "skills": []
}
```

Invoking the agent[#]

You can invoke the agent in several ways, including the kagent dashboard, kagent CLI, and the A2A host CLI.

Dashboards[#]

Launch the dashboard with `kagent dashboard`, find your research-crew, and start chatting. For complete steps, see the [Your First Agent](#) guide.

kagent CLI[#]

To use the kagent CLI, make sure that the controller is still being port-forwarded.

Then, use the `invoke` command. For more options, run `kagent help invoke`.

kagent invoke --agent research-crew --task "Research topics on AI reliability and provide a summary."

Example output: The output includes both the response as well as the details of the response. The formatting is in JSON but can be quite long, depending on the call and the agent configuration.

```
{
  "artifacts": [
    {
      "artifactId": "2d44a62e-d079-4dae-8ee8-f8759add9ffe",
      "parts": [
        {
          "kind": "text",
          "text": "After researching AI reliability, I found that it encompasses aspects like robustness,
safety, and trustworthiness..."
        }
      ]
    }
  ],
  ...
}
```

A2A host CLI#

You can use the A2A host CLI to invoke the agent. This CLI is part of the [A2A samples repository](#).

1. Clone the A2A samples repository.

```
git clone https://github.com/a2aproject/a2a-samples.git
```

2. From the a2a-samples/samples/python/hosts/cli directory, point the CLI to the kagent endpoint.

```
cd a2a-samples/samples/python/hosts/cli
```

```
uv run . --agent http://127.0.0.1:8083/api/a2a/kagent/research-crew
```

Example output: The CLI connects to the kagent, displays the agent card and prompts you for input.

```
===== Agent Card =====
```

```
{"capabilities":{"pushNotifications":false,"stateTransitionHistory":true,"streaming":true},"defaultInputModes":["text"],"defaultOutputModes":["text"],"description":"A research crew with multiple specialized agents for web research and
```

```
analysis.", "name": "research_crew", "protocolVersion": "0.2.6", "skills": [], "url": "http://127.0.0.1:8083/api/a2a/kagent/research-crew/", "version": ""}
```

===== starting a new task =====

What do you want to send to the agent? (:q or quit to exit):

3. Send the task "Research topics on AI reliability and provide a summary." to the agent. You'll be also prompted to optionally attach a file to the request, but just hit enter to skip this step.

Example output: You get a stream of events that include the prompt and the agent's response, such as the following.

```
{
  "contextId": "157a0834df2c459d9cee45316fffb5b",
  "final": false,
  "kind": "status-update",
  "metadata": {
    "crewai_app_name": "kagent__NS__research_crew",
    "crewai_author": "research_agent",
    "crewai_invocation_id": "e-8619b200-2f0a-4257-bd6b-b08bd1b139fd",
    "crewai_session_id": "157a0834df2c459d9cee45316fffb5b",
    "crewai_usage_metadata": {
      "candidatesTokenCount": 150,
      "candidatesTokensDetails": [
        {
          "modality": "TEXT",
          "tokenCount": 150
        }
      ],
      "promptTokenCount": 500,
      "promptTokensDetails": [
        {
          "modality": "TEXT",
          "tokenCount": 500
        }
      ]
    }
  }
}
```

```
    }
  ],
  "totalTokenCount": 650
},
"crewai_user_id": "admin@kagent.dev"
},
"status": {
  "message": {
    "kind": "message",
    "messageId": "dd05c3cd-2dc3-4efd-9791-d7124be6dd52",
    "parts": [
      {
        "kind": "text",
        "text": "After researching AI reliability, I found that it encompasses aspects like robustness,
safety, and trustworthiness..."
      }
    ],
    "role": "agent"
  },
  "state": "working",
  "timestamp": "2025-08-14T22:15:04.276358+00:00"
},
"taskId": "59d2b071-04e9-4fef-a0dd-e925dd13cceb"
}
```